

Spring Framework — Day 15 Notes

Topic: Bean Scope, Hashcode & the Singleton Design Pattern

1. Hashcode — A Quick Recap

A **hashcode** is a unique identification number assigned to a particular object. It helps distinguish one object in memory from another.

Consider the following two objects created using the **new** keyword:

```
A a1 = new A();
A a2 = new A();

System.out.println(a1.hashCode() + " " + a2.hashCode());

// Output: hashCode will be DIFFERENT
```

Every time the **new** keyword is used, a fresh object is created in memory, so **a1** and **a2** will have different hashcodes.

What happens when Spring's IoC container creates the bean?

If we ask the Spring IoC container to return the object of our class, and we call **getBean()** multiple times, does it return the same object or a different object each time?

- **Answer:** The same object is returned.
- This happens because, by default, the Spring IoC container creates beans with **singleton scope** — meaning only one instance of the bean is created for the entire container.

```
A bean = context.getBean("a", A.class);
A bean2 = context.getBean("a", A.class);

System.out.println(bean.hashCode() + " " + bean2.hashCode());

// Output: hashCode is the SAME for both
```

Since both references point to the same object, their hashcodes will match. This is the core behavior of Spring's default singleton bean scope.

2. Understanding the Singleton Pattern

Before we go further into Spring's bean scopes, let us understand the **Singleton Design Pattern** on which the default bean scope is based.

The word **singleton** means “only one object should be created” for a class throughout the application. However, a normal Java class does not behave this way by default — any code that has access to the class can create as many objects as it wants using the **new** keyword:

```
class SingletonApplication {

public SingletonApplication() {
// a public constructor allows unlimited
// object creation from anywhere
}
}
```

To truly restrict a class to a single instance, we need to follow a set of rules while designing it.

Rules to make a class Singleton

- **Rule 1:** Stop other classes from creating objects of this class directly.
- **Rule 2:** Only this class itself should be allowed to create its object.
- **Rule 3:** Create one method that internally creates and returns the single object of this class.
- **Rule 4:** This method should be **static**, so that it can be accessed without first creating an object of the class.

Singleton Class — Sample Code

```
class SingletonApplication {

// Rule: a private static reference holds the
// one and only instance of the class
private static SingletonApplication ref = null;

// Rule: private constructor blocks object
// creation from outside the class
private SingletonApplication() {
}

// Rule: public static method controls
// object creation and access
public static SingletonApplication getObject() {

// Create the object only if it does not
// already exist
if (ref == null) {
ref = new SingletonApplication();
}
return ref;
}
}
```

Notice that the constructor is marked **private**, so no other class can use the **new** keyword to create an object of **SingletonApplication**. The only way to get the object is through the static **getObject()** method, which creates the instance the first time it is called and returns the same instance on every later call.

Proof — Calling from Two Different Classes

```

class Application1 {
public void fetchObject() {
SingletonApplication ref1 = SingletonApplication.getObject();
System.out.println(ref1.hashCode());
}
}

class Application2 {
public void fetchObject() {
SingletonApplication ref1 = SingletonApplication.getObject();
System.out.println(ref1.hashCode());
}
}

public class Main {
public static void main(String[] args) {

Application1 app1 = new Application1();
Application2 app2 = new Application2();

app1.fetchObject();
app2.fetchObject();
}
}

```

Even though **Application1** and **Application2** are two completely different classes, both will print the **same hashcode**. This proves that only one object of **SingletonApplication** was ever created and it was shared by both classes.

3. A Practical Example — Database Connection

A common real-world use case for the Singleton pattern is managing a database connection. Creating a new connection every time is expensive, so it makes sense to create it only once and reuse it.

```

class ConnectionProvider {

private static Connection con = null;

public static Connection getConnection() {

if (con == null) {
Class.forName("jdbc.oracle.driver.OracleDriver");
con = DriverManager.getConnection(
"jdbc:oracle:thin:@localhost:1521:xe",
"system", "manager");
}
return con;
}
}

```

4. When Should a Class Be Made Singleton?

The decision of whether to make a class Singleton depends on whether the class holds any **instance (member) variables** that differ from object to object.

Case 1 — Class has instance variables (Singleton NOT suitable)

If a class stores data that is unique to every object, then creating multiple objects is necessary and correct. Making such a class Singleton would be wrong.

```
class Rectangle {
    int length;
    int width;

    Rectangle(int length, int width) {
        this.length = length;
        this.width = width;
    }

    public void getArea() {
        System.out.println(length * width);
    }
}

// Each object holds its own length and width,
// so separate objects are needed
Rectangle person1 = new Rectangle(10, 20);
Rectangle person2 = new Rectangle(30, 40);
```

Case 2 — Class has no instance variables (Singleton is suitable)

If a class has no member variables at all, then every object created from it would be functionally identical and empty. Creating multiple such objects is unnecessary and wasteful of memory. In this case, it is better to make the methods **static** so the class can behave like a Singleton.

```
class Rectangle {

    private Rectangle() {
    }

    public static void getArea(int length, int width) {
        System.out.println(length * width);
    }
}
```

Instead of writing this (unnecessary object creation):

```
Rectangle person1 = new Rectangle();
person1.getArea(10, 20);

Rectangle person2 = new Rectangle();
person2.getArea(30, 40);
```

We can simply write this:

```
Rectangle.getArea(10, 20);  
Rectangle.getArea(30, 40);
```

Rule of thumb: If a class has no state (no instance variables) to store per object, prefer static methods or the Singleton pattern instead of creating multiple objects.

End of Day 15 notes. Next session: more on Singleton and Spring bean scopes.